



React.js

Complete Beginner's Notes

JSX · Components · Hooks · Routing · Context · Performance

Your definitive starting guide to modern React development

Table of Contents

No.	Topic
1	Introduction to React
2	JSX Syntax
3	Components – Functional & Class
4	Props
5	State & useState Hook
6	Lifecycle & useEffect Hook
7	Event Handling
8	Conditional Rendering
9	Lists & Keys
10	Forms & Form Handling
11	React Router Basics
12	Context API
13	Additional Hooks
14	Performance Optimization
15	Tooling & Best Practices

■ 1. Introduction to React

React is an open-source JavaScript **UI library** created by Facebook (Meta) in 2013. It lets you build fast, interactive user interfaces by breaking the UI into small, reusable pieces called **components**.

■ Why React?

- **Component-based** – build once, reuse everywhere.
- **Virtual DOM** – React keeps a lightweight copy of the real DOM and only updates what changed, making apps fast.
- **Declarative** – you describe *what* the UI should look like; React figures out *how* to update it.
- **Huge ecosystem** – massive community, libraries, and job market demand.

■ How React works (big picture)

1. You write **components** (JavaScript functions/classes that return JSX).
2. React builds a **Virtual DOM** tree from those components.
3. When data changes, React compares old vs new Virtual DOM (**diffing**).
4. Only the changed parts are updated in the **real DOM** (reconciliation).

■ Setting Up – Create React App

```
# Install Node.js first (nodejs.org), then run:
npx create-react-app my-app
cd my-app
npm start           # Opens http://localhost:3000
```

■ **TIP:** Use **Vite** for faster builds: `npx create-vite@latest my-app --template react`

■ 2. JSX Syntax

JSX (JavaScript XML) is a syntax extension that lets you write HTML-like code inside JavaScript. Browsers don't understand JSX directly — **Babel** compiles it into regular JavaScript.

■ Key JSX Rules

- Every component must return a **single root element** (wrap siblings in `<div>` or `<> ... </>`).
- Use **className** instead of `class`; **htmlFor** instead of `for`.
- All tags must be **self-closing** or properly closed: `<img />`, `<br />`.
- JavaScript expressions go inside **{ curly braces }**.
- Inline styles use a JavaScript **object**: `style={{ color: 'red' }}`.

```
// JSX Example
function Greeting() {
  const name = 'Pramuditha';
  const isStudent = true;
  return (
    <div className='greeting'>
      <h1>Hello, {name}!</h1>
      <p>{isStudent ? 'Student ■' : 'Professional ■'}</p>
      <img src='avatar.png' alt='avatar' />
    </div>
  );
}
```

■ ■ **PITFALL:** Never put statements (if, for) directly inside JSX. Use ternary operators or move logic above the return.

■ 3. Components – Functional & Class

A **component** is a self-contained piece of UI — like a LEGO brick. React has two types: **Functional** (modern, preferred) and **Class** (legacy).

■ Functional Component (■ Recommended)

```
// Arrow function component
const Welcome = () => {
  return <h2>Welcome to React! ■</h2>;
};

// Short form (implicit return)
const Badge = () => <span className='badge'>New</span>;
```

■ Class Component (Legacy — still good to know)

```
import React, { Component } from 'react';

class Welcome extends Component {
  render() {
    return <h2>Welcome, {this.props.name}</h2>;
  }
}
```

■ **TIP:** Always start component names with a **Capital Letter**. React treats lowercase tags as HTML elements.

■ ■ **PITFALL:** Never define a component inside another component — it causes unnecessary re-creation on each render.

■ 4. Props (Properties)

Props are how you pass data from a **parent** component to a **child** component. Think of them like function arguments. Props are **read-only** — a child can never modify its own props.

```
// Child component receives props
const UserCard = ({ name, role, avatar }) => (
  <div className='card'>
    <img src={avatar} alt={name} />
    <h3>{name}</h3>
    <p>{role}</p>
  </div>
);

// Parent passes props
const App = () => (
  <UserCard
    name='Pramuditha'
    role='CS Student'
    avatar='user.jpg'
  />
);
```

■ Default Props & PropTypes

```
import PropTypes from 'prop-types';

const Button = ({ label = 'Click Me', color = 'blue' }) => (
  <button style={{ background: color }}>{label}</button>
);

Button.propTypes = {
  label: PropTypes.string,
  color: PropTypes.string,
};
```

■■ **PITFALL:** Props flow only **one way** — parent → child. To send data upward, pass a callback function as a prop.

■ 5. State & useState Hook

State is data that a component manages itself and that can change over time. When state changes, React automatically **re-renders** the component to reflect the new value.

```
import { useState } from 'react';

const Counter = () => {
  // [currentValue, setterFunction] = useState(initialValue)
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>■ Add</button>
      <button onClick={() => setCount(count - 1)}>■ Sub</button>
      <button onClick={() => setCount(0)}>■ Reset</button>
    </div>
  );
};
```

■ Object State

```
const [user, setUser] = useState({ name: '', age: 0 });

// Always spread old state when updating one field
setUser(prev => ({ ...prev, name: 'Pramuditha' }));
```

■■ **PITFALL:** Never mutate state directly (e.g., state.count++). Always use the setter function — direct mutation won't trigger a re-render.

■ **TIP:** State updates are **asynchronous**. Use the functional update form setCount(prev => prev + 1) when the new value depends on the old one.

■■ 6. Lifecycle Methods & useEffect

Every component goes through a **lifecycle**: Mount → Update → Unmount. In functional components, the **useEffect** hook handles side effects (API calls, timers, subscriptions) at the right lifecycle phase.

```
import { useState, useEffect } from 'react';

const FetchUser = () => {
  const [user, setUser] = useState(null);

  // Runs ONCE after first render (componentDidMount)
  useEffect(() => {
    fetch('https://api.example.com/user/1')
      .then(res => res.json())
      .then(data => setUser(data));

    // Cleanup (componentWillUnmount)
    return () => console.log('Cleaned up!');
  }, []); // <-- empty array = run only once

  return <div>{user ? user.name : 'Loading...'}</div>;
};
```

■ Dependency Array Cheat-Sheet

■■ **PITFALL:** Forgetting the dependency array causes an infinite loop — the effect runs, updates state, triggers re-render, runs again...

■ 7. Event Handling

React handles events similarly to HTML but with **camelCase** names and you pass a **function reference**, not a string.

```
const Form = () => {
  const handleClick = () => alert('Button clicked!');
  const handleChange = (e) => console.log(e.target.value);
  const handleSubmit = (e) => {
    e.preventDefault(); // Stop page reload
    console.log('Submitted!');
  };

  return (
    <form onSubmit={handleSubmit}>
      <input onChange={handleChange} placeholder='Type here' />
      <button onClick={handleClick}>Click Me</button>
      <button type='submit'>Submit</button>
    </form>
  );
};
```

■ **PITFALL:** Don't call the function in JSX: `onClick={handleClick()}` — this runs immediately on render. Pass the reference: `onClick={handleClick}`.

■ 8. Conditional Rendering

Show or hide UI elements based on conditions — just like an if/else in JavaScript, but used inside JSX.

```
const Dashboard = ({ isLoggedIn, username, notifications }) => {
  // 1. if/else (outside JSX)
  if (!isLoggedIn) return <p>Please log in.</p>;

  return (
    <div>
      {/* 2. Ternary operator */}
      <h2>{isLoggedIn ? `Hi, ${username}!` : 'Guest'}</h2>

      {/* 3. && short-circuit (render only if truthy) */}
      {notifications > 0 && <span>■ {notifications} new</span>}
    </div>
  );
};
```

■ **TIP:** Use the **ternary** operator for simple show/hide. Use **&&** when you only need to render something (no else branch).

■ 9. Lists & Keys

Use the **.map()** method to render arrays of items. Every item in a list must have a unique **key** prop — this helps React identify which items changed, added, or removed.

```
const students = [
  { id: 1, name: 'Pramuditha', grade: 'A' },
  { id: 2, name: 'Kasun', grade: 'B' },
```

```

    { id: 3, name: 'Nimali',    grade: 'A+'},
  ];

const StudentList = () => (
  <ul>
    {students.map(student => (
      <li key={student.id}>    {/* key must be unique & stable */}
        {student.name} – {student.grade}
      </li>
    ))}
  </ul>
);

```

■ **PITFALL:** Never use array **index** as a key if the list can be reordered or filtered — it causes bugs. Use a unique ID from your data instead.

■ 10. Forms & Form Handling

In React, forms are usually **controlled components** — the input's value is tied to state, giving React full control over the form data.

```

import { useState } from 'react';

const LoginForm = () => {
  const [form, setForm] = useState({ email: '', password: '' });

  const handleChange = (e) => {
    setForm(prev => ({ ...prev, [e.target.name]: e.target.value }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Login:', form);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name='email' value={form.email}
        onChange={handleChange} placeholder='Email' />
      <input name='password' value={form.password} type='password'
        onChange={handleChange} placeholder='Password' />
      <button type='submit'>Login</button>
    </form>
  );
};

```

■ **TIP:** For complex forms consider libraries like **React Hook Form** or **Formik** — they handle validation and reduce boilerplate.

11. React Router Basics

React Router (v6) lets you add navigation and multiple pages to a single-page app (SPA) without full page reloads.

Install

```
npm install react-router-dom
```

Basic Setup

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

const Home = () => <h1>■ Home Page</h1>;
const About = () => <h1>■ About Page</h1>;
const Profile = () => <h1>■ Profile Page</h1>;

const App = () => (
  <BrowserRouter>
    <nav>
      <Link to='/'>Home</Link> |
      <Link to='/about'>About</Link> |
      <Link to='/profile'>Profile</Link>
    </nav>
    <Routes>
      <Route path='/' element={<Home />} />
      <Route path='/about' element={<About />} />
      <Route path='/profile' element={<Profile />} />
      <Route path='*' element={<h1>404 Not Found</h1>} />
    </Routes>
  </BrowserRouter>
);
```

Useful Hooks (React Router v6)

- **useNavigate()** – programmatically navigate: `navigate('/home')`
- **useParams()** – get URL params from `path='/user/:id'`
- **useLocation()** – access current URL info

■■ **PITFALL:** Use `<Link>` instead of `<a href>` for internal navigation — plain anchor tags cause a full page reload.

12. Context API

The **Context API** solves *prop drilling* — passing props through many layers of components. Context lets you share data (theme, user, language) globally without passing props at every level.

```
import { createContext, useContext, useState } from 'react';

// 1. Create Context
const ThemeContext = createContext();

// 2. Provider – wraps the component tree
const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = useState('light');
  const toggle = () => setTheme(t => t === 'light' ? 'dark' : 'light');
  return (
    <ThemeContext.Provider value={{ theme, toggle }}>
```



```

    {children}
  </ThemeContext.Provider>
);
};

// 3. Consumer — any nested component can read context
const ThemeButton = () => {
  const { theme, toggle } = useContext(ThemeContext);
  return <button onClick={toggle}>Current: {theme}</button>;
};

// 4. Wrap App with Provider
const App = () => (
  <ThemeProvider>
    <ThemeButton />
  </ThemeProvider>
);

```

■ **PITFALL:** Context re-renders ALL consumers when its value changes. For high-frequency updates, consider **Zustand** or **Redux Toolkit** instead.

■ 13. Additional Hooks

■ **useRef**

Access DOM elements or persist a value without re-rendering.

```

const inputRef = useRef(null);
// Focus input on button click:
<input ref={inputRef} />
<button onClick={() => inputRef.current.focus()}>Focus</button>

```

■ **useReducer**

Manage complex state logic — like `useState` but with a reducer function (similar to `Redux`).

```

const reducer = (state, action) => {
  switch(action.type) {
    case 'inc': return { count: state.count + 1 };
    case 'dec': return { count: state.count - 1 };
    default:   return state;
  }
};

const [state, dispatch] = useReducer(reducer, { count: 0 });
<button onClick={() => dispatch({ type: 'inc' })}>+</button>

```

■ **useMemo**

Cache an expensive calculation — only re-runs when dependencies change.

```

const total = useMemo(() => {
  return items.reduce((sum, item) => sum + item.price, 0);
}, [items]); // Only recalculate when items changes

```

■ **useCallback**

Cache a function reference — prevents unnecessary child re-renders.

```
const handleAdd = useCallback(() => {  
  setCount(c => c + 1);  
}, []); // Stable reference – won't change between renders
```

■ 14. Performance Optimization

React is fast by default, but as apps grow, optimization helps keep things smooth.

■ React.memo — Prevent Unnecessary Re-renders

```
// Without memo: re-renders every time parent re-renders
// With memo: only re-renders if props actually change
const ExpensiveChild = React.memo(({ data }) => {
  console.log('Child rendered');
  return <div>{data}</div>;
});
```

■ Code Splitting with React.lazy

```
import { lazy, Suspense } from 'react';

// Load component only when needed (reduces initial bundle size)
const HeavyPage = lazy(() => import('./HeavyPage'));

const App = () => (
  <Suspense fallback=<div>Loading page...</div>>
    <HeavyPage />
  </Suspense>
);
```

■ Quick Performance Tips

- Use **keys** properly in lists — avoids unnecessary DOM re-creation.
- Avoid **anonymous functions** in JSX props — use `useCallback` instead.
- Keep state as **local as possible** — don't lift state higher than needed.
- Use **React DevTools Profiler** to identify slow components.
- Paginate or **virtualize** long lists with **react-window**.

■■ 15. Tooling & Best Practices

■ Essential Tools & Libraries

Tool / Library	Purpose
Create React App	Zero-config project scaffold (beginner-friendly)
Vite	Ultra-fast dev server & build tool (recommended)
React DevTools	Browser extension — inspect component tree & state
ESLint + Prettier	Code linting & automatic formatting
React Router DOM	Client-side routing / navigation
Axios / Fetch	HTTP requests to APIs
React Hook Form	Performant form handling with validation
Tailwind CSS	Utility-first CSS framework
Zustand / Redux TK	Global state management (beyond Context)

■ Best Practices Checklist ■

- One component per file; name the file same as the component.
- Keep components **small and focused** — single responsibility.
- Lift state up only when two components need the same data.
- Use **custom hooks** (use... prefix) to share logic between components.
- Always handle **loading** and **error** states in async operations.
- Write **PropTypes** or use **TypeScript** to catch type errors early.
- Never put **side effects** directly in render — always use `useEffect`.
- Prefer **const** over `let`; avoid `var` entirely.

■ **Keep Building!** The best way to learn React is to build projects. Start small — a to-do app, a weather app — and grow from there. You've got this! ■■